

Produit final fonctionnel – Réalisation du projet

Titre du projet : Prédiction de la réussite scolaire des étudiants

Étudiant : Laurence Leduc Lacaille

Superviseur : Fatima Bensalma

Domaines : Science des données

1. Introduction

Le projet porte sur la modélisation et la prédiction de la réussite scolaire à partir du jeu de données *Student Exam Performance Dataset* (Kaggle, s. d.), qui regroupe des variables académiques, comportementales et socio-démographiques décrivant le profil des étudiants. L'objectif général est de développer et de comparer plusieurs modèles de classification afin d'évaluer leur performance dans la prédiction de la réussite scolaire et d'identifier les facteurs influençant les résultats académiques.

Trois hypothèses guident cette démarche. La première (H1) stipule que le modèle de forêt aléatoire offrira une performance prédictive supérieure à celle des autres modèles, notamment la régression logistique et les arbres de décision. La deuxième (H2) suppose que les variables liées à l'environnement scolaire des étudiants auront un impact plus important sur la réussite que les variables socio-démographiques. Enfin, la troisième (H3) avance que les performances extrêmes sont associées à des profils spécifiques d'étudiants, caractérisés par certaines combinaisons de variables.

Le présent document a pour rôle de décrire la mise en œuvre technique du projet, en détaillant les différentes étapes de traitement et d'analyse réalisées à l'aide du langage R. Il expose la démarche méthodologique suivie, les transformations appliquées aux données, la construction des modèles de régression logistique, d'arbres de décision et de forêt aléatoire, ainsi que les procédures de validation et de comparaison employées. Les résultats seront présentés à la suite de l'implémentation de la méthode, suivis de leur interprétation et des conclusions.

2. Outils et environnement de développement

L'ensemble des analyses a été effectué à l'aide du logiciel R, en utilisant l'environnement de développement RStudio. Les visualisations ont été réalisées grâce à l'outil Shiny, qui permet de créer des interfaces interactives facilitant l'exploration des données, l'analyse des relations entre les variables ainsi que la présentation des résultats issus des modèles de classification.

Shiny repose sur une architecture divisée en deux composantes principales : une interface utilisateur (UI), qui permet l'affichage et l'interaction avec les graphiques, et un serveur (server), qui gère les traitements et la génération dynamique des visualisations à partir des données. Dans ce projet, cette structure permet de produire les graphiques en fonction des variables sélectionnées, facilitant ainsi l'exploration des données.

La fonction responsable de l’affichage des graphiques se nomme *dashboard*. Développée avec Shiny, elle permet de générer les différentes visualisations à partir des fonctions qui seront présentées dans les sections suivantes. Le tableau de bord généré sera présenté dans le prochain rapport, soit le rapport final. Les graphiques seront toutefois présentés individuellement pour l’analyse des résultats. Le code de la fonction *dashboard* est présenté en annexe.

Enfin, le projet est structuré et déposé sur la plateforme GitHub, afin d’assurer le partage du travail et la reproductibilité des analyses.

Lien vers le projet : <https://github.com/lau1258/Modelisation-R-ussite-Scolaire.git>

3. Implémentation de la méthode

Cette section présente la démarche méthodologique adoptée dans le cadre de ce projet, incluant la préparation des données, la modélisation, l’évaluation des modèles ainsi que l’analyse complémentaire. Les graphiques générés par les différentes fonctions présentées dans cette section seront exposés et interprétés dans la section suivante consacrée aux résultats.

3.1 Prétraitement et nettoyage des données

Dans un premier temps, deux fonctions ont été créées afin de résumer rapidement les variables numériques et qualitatives. La fonction `resume_num()` sélectionne automatiquement les colonnes numériques du jeu de données et calcule, pour chacune d’elles, le minimum, la moyenne, le maximum, le nombre de valeurs manquantes ainsi que le pourcentage de valeurs égales à zéro. Cette fonction permet donc de repérer rapidement des valeurs aberrantes, des valeurs manquantes ou des valeurs inhabituelles dans les variables quantitatives. Le code est le suivant :

```
#-----  
# 1.1.1 Résumé des variables numériques  
#-----  
resume_num <- function(table) {  
  # Sélection des colonnes numériques  
  num_data <- table[sapply(table, is.numeric)]  
  
  # Construction du résumé  
  res <- data.frame(  
    Min = sapply(num_data, function(v) round(min(v, na.rm = TRUE), 1)),  
    Moyenne = sapply(num_data, function(v) round(mean(v, na.rm = TRUE), 1)),  
    Max = sapply(num_data, function(v) round(max(v, na.rm = TRUE), 1)),  
    NAs = sapply(num_data, function(v) sum(is.na(v))),  
  
    zeros = sapply(num_data, function(v) {  
      round(sum(v == 0, na.rm = TRUE) / sum(!is.na(v)) * 100, 1)  
    }),  
  
    stringsAsFactors = FALSE  
  )  
}
```

```
    return(res)
}
```

Ensuite, la fonction `resume_quali()` permet d'examiner les variables qualitatives. Elle parcourt les colonnes de type texte ou facteur, affiche les modalités présentes dans chaque variable et compte les valeurs manquantes. Dans ce code, les chaînes vides sont aussi considérées comme des valeurs manquantes grâce à `trimws(x) == ""`. Cela est important, car certaines valeurs absentes ne sont pas toujours codées comme `NA`, mais peuvent plutôt apparaître comme des champs vides. Le code est le suivant :

```
#-----
# 1.1.2 Résumé des variables qualitatives
#-----
resume_quali <- function(df) {

  for (var in names(df)) {

    if (is.factor(df[[var]]) | is.character(df[[var]])) {

      cat("\n-----\n")
      cat("Variable :", var, "\n")

      x <- as.character(df[[var]])

      # Nombre de NA et chaînes vides
      nas <- sum(is.na(x) | trimws(x) == "")

      # Modalités visibles avec guillemets
      modalities <- unique(x)
      modalities_affichage <- paste0("'", modalities, "'")

      cat("Modalités :", paste(modalities_affichage, collapse = ", "), "\n")
      cat("Valeurs manquantes :", nas, "\n")
    }
  }
}
```

Les données ont ensuite été importées à partir du fichier CSV. Seules les variables retenues pour le projet ont été conservées. Pour les variables numériques, on retrouve : `Sleep_Hours`, `Physical_Activity`, `Hours_Studied`, `Attendance`, `Previous_Scores` et `Exam_Score` (qui constitue la variable cible initiale et qui sera transformée pour la modélisation). Pour les variables qualitatives, on retrouve : `Motivation_Level`, `Gender`, `Access_to_Resources`, `Parental_Education_Level` et `Parental_Involvement`. Une nouvelle variable binaire, nommée *Success*, a été créée à partir de *Exam_Score* et sera utilisée comme variable cible pour la modélisation. Cette variable a été construite selon la condition suivante : les étudiants ayant obtenu une note de 60 ou plus sont classés dans la catégorie *Success*, tandis que ceux ayant obtenu une note inférieure à 60 sont classés dans la catégorie *Failure*. Le code qui suit illustre ces explications.

```
# Importer les données depuis le fichier CSV
data <- read.csv(
  "StudentPerformanceFactors.csv",
```

```
header = TRUE,
sep = ", "
)

# Garder les colonnes sélectionnées seulement
data <- data %>%
  select(
    Parental_Involvement,
    Parental_Education_Level,
    Access_to_Resources,
    Gender,
    Sleep_Hours,
    Motivation_Level,
    Physical_Activity,
    Hours_Studied,
    Attendance,
    Previous_Scores,
    Exam_Score
  )

# Création d'une variable binaire Success/Failure à partir de Exam_Score
data$Success <- factor(
  ifelse(data$Exam_Score >= 60, "Success", "Failure")
)
```

La commande `str(data)` a permis de vérifier la structure du jeu de données après l'importation. Le jeu de données contient 6607 observations et 12 variables. Les variables numériques, comme `Sleep_Hours`, `Physical_Activity`, `Hours_Studied`, `Attendance`, `Previous_Scores` et `Exam_Score`, sont bien reconnues comme des entiers (`int`). Par contre, les variables qualitatives sont importées comme des chaînes de caractères (`chr`). Elles devront donc être transformées en facteurs afin d'être correctement interprétées par les modèles de classification. Le tableau suivant illustre le résultat de la commande `str(data)`.

Code :

```
# Vérifier le type des données
str(data)
```

Résultats :

```
'data.frame':    6607 obs. of  12 variables:
 $ Parental_Involvement    : chr  "Low" "Low" "Medium" "Low" ...
 $ Parental_Education_Level: chr  "High School" "College" "Postgraduate" "High
School" ...
 $ Access_to_Resources     : chr  "High" "Medium" "Medium" "Medium" ...
 $ Gender                  : chr  "Male" "Female" "Male" "Male" ...
 $ Sleep_Hours             : int   7 8 7 8 6 8 7 6 6 8 ...
 $ Motivation_Level        : chr  "Low" "Low" "Medium" "Medium" ...
 $ Physical_Activity       : int   3 4 4 4 4 3 2 2 1 5 ...
 $ Hours_Studied           : int  23 19 24 29 19 19 29 25 17 23 ...
 $ Attendance              : int  84 64 98 89 92 88 84 78 94 98 ...
 $ Previous_Scores         : int  73 59 91 98 65 89 68 50 80 71 ...
 $ Exam_Score              : int   67 61 74 71 70 71 67 66 69 72 ...
 $ Success                  : Factor w/ 2 levels "Failure","Success": 2 2 2 2 2 2
2 2 2 2 ...
```

Le résumé des variables numériques ne montre aucune valeur manquante. Les valeurs observées semblent également cohérentes avec le contexte des données. Par exemple, le nombre d'heures de sommeil varie entre 4 et 10, l'assiduité varie entre 60 et 100, et les résultats antérieurs varient entre 50 et 100. La variable `Physical_Activity` contient quelques valeurs égales à zéro, mais cela correspond aux étudiants ne pratiquant aucune activité physique, plutôt qu'à une erreur de saisie. Le tableau suivant illustre le résultat de la fonction `resume_num(data)`.

Code :

```
resume_num(data)
```

Résultats :

	Min	Moyenne	Max	NAs	zeros
Sleep_Hours	4	7.0	10	0	0.0
Physical_Activity	0	3.0	6	0	0.7
Hours_Studied	1	20.0	44	0	0.0
Attendance	60	80.0	100	0	0.0
Previous_Scores	50	75.1	100	0	0.0
Exam_Score	55	67.2	101	0	0.0

Pour les variables qualitatives, le résumé montre que la majorité des variables ne contiennent aucune valeur manquante. Cependant, la variable `Parental_Education_Level` contient une modalité vide "", ce qui correspond à 90 valeurs manquantes. Cette observation justifie une étape de nettoyage supplémentaire afin de retirer les lignes concernées avant de poursuivre les analyses. Le tableau suivant illustre le résultat de la fonction `resume_quali(data)`.

Code :

```
resume_quali(data)
```

Résultats :

```
-----
Variable : Parental_Involvement
Modalités : "Low", "Medium", "High"
Valeurs manquantes : 0
-----
Variable : Parental_Education_Level
Modalités : "High School", "College", "Postgraduate", ""
Valeurs manquantes : 90
-----
Variable : Access_to_Resources
Modalités : "High", "Medium", "Low"
Valeurs manquantes : 0
-----
Variable : Gender
Modalités : "Male", "Female"
Valeurs manquantes : 0
-----
```

```
Variable : Motivation_Level
Modalités : "Low", "Medium", "High"
Valeurs manquantes : 0
-----
Variable : Success
Modalités : "Success", "Failure"
Valeurs manquantes : 0
```

Finalement, les lignes contenant une valeur vide dans la variable `Parental_Education_Level` ont été supprimées, car ces valeurs manquantes (chaînes vides) auraient pu entraîner des erreurs ou des incohérences dans les analyses et la modélisation, notamment lors de la conversion en facteurs et de l'entraînement des modèles. Ensuite, les variables qualitatives ont été converties en facteurs. Pour les variables ordinales, comme `Motivation_Level`, `Access_to_Resources` et `Parental_Involvement`, les niveaux ont été définis dans un ordre logique allant de `Low` à `High`. Pour `Parental_Education_Level`, l'ordre a été défini comme suit : `High School`, `College`, `Postgraduate`. La variable `Gender`, quant à elle, a été convertie en facteur simple. Cette conversion permettra par la suite d'assurer un affichage cohérent des catégories selon l'ordre souhaité. Les lignes de code qui suivent illustrent ces explications.

```
# Supprimer les lignes avec valeur vide de Parental_Education_Level
data <- data[trimws(data$Parental_Education_Level) != "", ]

# Mettre les variables qualitatives en factor
data$Motivation_Level <- factor(data$Motivation_Level, levels = c("Low", "Medium",
"High"))

data$Access_to_Resources <- factor(data$Access_to_Resources, levels = c("Low",
"Medium", "High"))

data$Parental_Involvement <- factor(data$Parental_Involvement, levels = c("Low",
"Medium", "High"))

data$Parental_Education_Level <- factor(data$Parental_Education_Level, levels =
c("High School", "College", "Postgraduate"))

data$Gender <- as.factor(data$Gender)
```

3.2 Analyse exploratoire

Avant d'entraîner les modèles de prédiction, il est essentiel d'examiner la distribution et corrélation des variables afin de mieux comprendre leur comportement.

La fonction `histplot` permet de générer des histogrammes pour les variables numériques sélectionnées. Elle reçoit comme arguments le jeu de données, la liste des variables à représenter, le nombre de classes de l'histogramme et les étiquettes en français associées aux variables. Pour chaque variable, la fonction utilise `ggplot2` afin de représenter la distribution des valeurs sous forme d'histogramme. Cela permet d'observer la forme générale des données, par exemple si les valeurs sont concentrées autour d'un certain intervalle, si elles

sont dispersées ou si certaines valeurs semblent plus rares. Le code suivant illustre la fonction `histplot`.

```
# 1.2.1 Histogrammes - variables numériques
#-----
histplot <- function(data, var_names, bins = 30, labels) {

  plots <- lapply(var_names, function(var_name) {

    ggplot(data, aes(x = .data[[var_name]])) +
      geom_histogram(bins = bins, fill = "steelblue", color = "white") +
      labs(
        title = paste("Distribution de", labels[[var_name]]),
        x = labels[[var_name]],
        y = "Fréquence"
      ) +
      theme_minimal()
  })

  patchwork::wrap_plots(plots)
}
```

La fonction `barplot_quali` permet de générer des graphiques en barres pour les variables qualitatives. Elle reçoit comme arguments le jeu de données, les variables qualitatives à analyser, les étiquettes en français des variables, les étiquettes des modalités et le nombre de colonnes souhaité pour l’affichage des graphiques. Pour chaque variable, la fonction produit un graphique de fréquences, c’est-à-dire qu’elle affiche le nombre d’observations dans chaque catégorie. Les valeurs de fréquence sont aussi ajoutées directement au-dessus des barres, ce qui facilite la lecture des résultats. Cette fonction est particulièrement utile pour vérifier la répartition des catégories et repérer si certaines modalités sont beaucoup plus présentes que d’autres dans les données. Le code suivant illustre la fonction `barplot_quali`.

```
# 1.2.2 Barplots - variables qualitatives
#-----
barplot_quali <- function(data, var_names, labels, labels_levels, ncol = 2) {

  library(ggplot2)
  library(patchwork)

  plots <- lapply(var_names, function(var_name) {

    var_label <- ifelse(var_name %in% names(labels), labels[[var_name]], var_name)

    ggplot(data, aes(x = .data[[var_name]])) +
      geom_bar(fill = "steelblue") +
      stat_count(
        aes(label = after_stat(count)),
        geom = "text",
        vjust = -0.1
      ) +
      scale_x_discrete(labels = labels_levels) +
      scale_y_continuous(expand = expansion(mult = c(0, 0.1))) +
      labs(
        title = paste("Distribution de fréquences -", var_label),
        x = var_label,
        y = "Fréquence"
      )
  })

  patchwork::wrap_plots(plots, ncol = ncol)
}
```

```
    ) +  
    theme_minimal()  
  })  
  
  patchwork::wrap_plots(plots, ncol = ncol)  
}
```

La fonction `correlation` permet d'analyser les relations entre les variables numériques à l'aide d'une matrice de corrélation. Elle sélectionne d'abord les variables numériques choisies, puis calcule les coefficients de corrélation entre celles-ci avec la fonction `cor`. Les noms des variables sont ensuite remplacés par leurs étiquettes en français afin de rendre le graphique plus clair. La matrice est finalement représentée sous forme de carte de chaleur avec `corrplot`, ce qui permet de visualiser rapidement la force et le sens des relations linéaires entre les variables numériques. Cette fonction permet donc de voir si certaines variables évoluent ensemble. Le code suivant illustre la fonction `correlation`.

```
# 1.3.1 Corrélation des variables numériques  
#-----  
correlation <- function(data, vars, labels) {  
  
  require(corrplot)  
  
  vars_df <- data[, vars, drop = FALSE]  
  cor_matrix <- cor(vars_df, use = "complete.obs")  
  
  colnames(cor_matrix) <- labels[colnames(cor_matrix)]  
  rownames(cor_matrix) <- labels[rownames(cor_matrix)]  
  
  col <- colorRampPalette(c("red", "green", "darkblue"))(200)  
  
  corrplot::corrplot(  
    cor_matrix,  
    method = "color",  
    type = "upper",  
    tl.col = "black",  
    tl.srt = 45,  
    col = col,  
    addCoef.col = "black",  
    number.cex = 0.8,  
    number.digits = 3,  
    diag = TRUE,  
    mar = c(0, 0, 1, 0),  
    is.corr = TRUE  
  )  
}
```

La fonction `cramer` permet d'analyser les relations entre les variables qualitatives à l'aide du V de Cramér. Cette mesure est adaptée aux variables catégorielles, puisqu'elle permet d'évaluer la force de l'association entre deux variables qualitatives. La fonction commence par sélectionner les variables choisies, puis les transforme en facteurs, si nécessaire, afin de s'assurer qu'elles soient bien traitées comme des variables catégorielles. Ensuite, une matrice vide est créée et remplie à l'aide de boucles qui calculent le V de Cramér pour chaque paire de variables. Lorsque la variable est comparée avec elle-même, la valeur est fixée à 1, car l'association est parfaite. Les résultats sont ensuite affichés dans une matrice visuelle avec

corrplot, ce qui permet d'identifier plus facilement les associations les plus fortes entre les variables qualitatives. Le code suivant illustre la fonction cramer.

```
# 1.3.2 Corrélation des variables qualitatives - V de Cramér
#-----
cramer <- function(data, vars, labels) {

  require(rcompanion)
  require(corrplot)

  df <- data[, vars, drop = FALSE]
  df[] <- lapply(df, as.factor)

  n <- ncol(df)
  mat <- matrix(NA, nrow = n, ncol = n)
  colnames(mat) <- colnames(df)
  rownames(mat) <- colnames(df)

  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        mat[i, j] <- 1
      } else {
        tab <- table(df[[i]], df[[j]])
        mat[i, j] <- suppressWarnings(rcompanion::cramerV(tab))
      }
    }
  }

  colnames(mat) <- labels[colnames(mat)]
  rownames(mat) <- labels[rownames(mat)]

  col <- colorRampPalette(c("green", "yellow", "red"))(200)

  corrplot::corrplot(
    mat,
    method = "color",
    type = "upper",
    tl.col = "black",
    tl.srt = 45,
    tl.cex = 0.8,
    col = col,
    addCoef.col = "black",
    number.cex = 0.75,
    number.digits = 3,
    diag = TRUE,
    mar = c(1, 1, 2, 1),
    is.corr = FALSE,
    col.lim = c(0, 1)
  )
}
```

3.3 Séparation des données

Avant de passer à la modélisation, les données ont été divisées en deux sous-ensembles : un ensemble d'entraînement (80 %) et un ensemble de test (20 %). Afin d'assurer la reproductibilité des résultats, une graine aléatoire a été fixée à l'aide de `set.seed(123)`. La

séparation a été réalisée de manière stratifiée avec la fonction `createDataPartition`, en se basant sur la variable cible *Success*. Cette approche permet de conserver la même proportion de classes (réussite/échec) dans les deux ensembles, ce qui est essentiel pour éviter un biais lié à un déséquilibre des classes lors de l'entraînement et de l'évaluation des modèles. Les observations sélectionnées ont été assignées à `data_train`, tandis que les autres ont été placées dans `data_test`, qui servira uniquement à évaluer la performance des modèles sur des données non vues. Le code suivant permet d'effectuer la séparation stratifiée des données en ensembles d'entraînement et de test.

```
# Préparation des données train/test
#-----
set.seed(123) # reproductibilité

# Split stratifié 80% train / 20% test
train_index <- createDataPartition(data$Success, p = 0.8, list = FALSE)

data_train <- data[train_index, ]
data_test  <- data[-train_index, ]
```

3.4 Modélisation

Dans cette étape, trois modèles de classification ont été construits afin de prédire la variable cible *Success* à partir des variables explicatives sélectionnées. Ces modèles ont été entraînés à l'aide de l'ensemble de données `data_train`, afin de pouvoir ensuite comparer leurs performances sur des données non vues.

3.4.1 Modélisation par régression logistique

Le modèle de régression logistique a été estimé à l'aide de la fonction `glm` avec l'argument `family = binomial`, ce qui permet de modéliser une variable dépendante binaire. Ce modèle vise à estimer la probabilité de réussite (*Success*) en fonction des variables explicatives. La régression logistique repose sur une relation linéaire entre les variables explicatives et le logit de la probabilité de succès, ce qui permet d'interpréter l'effet de chaque variable à l'aide des coefficients et des odds ratios. Le code suivant présente l'implémentation du modèle.

```
modele_logit <- glm(
  Success ~ Sleep_Hours + Physical_Activity + Hours_Studied +
    Attendance + Previous_Scores +
    Motivation_Level + Gender + Access_to_Resources +
    Parental_Education_Level + Parental_Involvement,
  data = data_train,
  family = binomial
)
```

Dans le cadre de l'évaluation des hypothèses du modèle de régression logistique, il est nécessaire de vérifier certaines conditions liées aux variables explicatives. La fonction `verif_logit` permet de vérifier visuellement l'hypothèse de linéarité du logit pour les variables numériques utilisées dans le modèle de régression logistique. La fonction commence par prédire les probabilités associées au modèle logistique à partir des données

fournies. Ces probabilités sont ensuite transformées en valeurs de logit. Pour éviter les erreurs de calcul lorsque les probabilités sont exactement égales à 0 ou à 1, elles sont légèrement ajustées. Ensuite, pour chaque variable indiquée, la fonction produit un graphique présentant la relation entre cette variable et le logit. Une droite de régression linéaire est ajoutée en rouge, tandis qu'une courbe lissée est ajoutée en bleu afin de comparer la tendance linéaire avec une tendance potentiellement non linéaire. Si les deux lignes suivent une forme similaire, cela suggère que l'hypothèse de linéarité du logit est raisonnablement respectée. Le code suivant suit cette logique.

```
verif_logit <- function(data, vars, modele, ncol = 2, labels) {  
  
  library(ggplot2)  
  library(patchwork)  
  
  p <- predict(modele, newdata = data, type = "response")  
  
  p[p == 0] <- 1e-6  
  p[p == 1] <- 1 - 1e-6  
  
  logit <- log(p / (1 - p))  
  
  plots <- lapply(vars, function(v) {  
  
    df <- data.frame(  
      x = data[[v]],  
      logit = logit  
    )  
  
    ggplot(df, aes(x = x, y = logit)) +  
      geom_point(alpha = 0.4) +  
      geom_smooth(method = "lm", color = "red", se = FALSE) +  
      geom_smooth(  
        method = "gam",  
        formula = y ~ s(x, k = 4),  
        color = "blue",  
        se = FALSE,  
        linewidth = 0.5  
      ) +  
      labs(  
        title = paste("Vérification du logit -", labels[[v]]),  
        x = labels[[v]],  
        y = "Logit"  
      ) +  
      theme_minimal()  
  })  
  
  patchwork::wrap_plots(plots, ncol = ncol)  
}
```

Ensuite, la fonction `plot_vif` permet de visualiser la multicollinéarité entre les variables explicatives du modèle. La fonction utilise d'abord la fonction `vif` du package `car` pour calculer les valeurs de VIF. Lorsque des variables qualitatives possèdent plusieurs modalités, le modèle peut produire un GVIF, c'est-à-dire une version généralisée du VIF. Dans ce cas, la fonction utilise le VIF ajusté afin de rendre les valeurs plus comparables entre les variables. Les noms des variables sont ensuite traduits à l'aide de la liste `labels`, ce qui permet

d'afficher des noms plus compréhensibles dans le graphique. Le graphique final présente les valeurs de VIF ajusté pour chaque variable, avec une ligne verticale pointillée à 5 servant de seuil de référence. Une valeur élevée peut indiquer un problème potentiel de multicollinéarité. Le code correspondant est le suivant.

```
plot_vif <- function(modele, labels) {

  library(car)
  library(ggplot2)
  library(dplyr)
  library(tibble)

  vif_values <- car::vif(modele)

  vif_df <- as.data.frame(vif_values) %>%
    rownames_to_column("Variable")

  # VIF ajusté
  if ("GVIF^(1/(2*Df))" %in% colnames(vif_df)) {
    vif_df <- vif_df %>%
      mutate(VIF_ajuste = `GVIF^(1/(2*Df))`)
  } else {
    vif_df <- vif_df %>%
      mutate(VIF_ajuste = GVIF)
  }

  # Traduction variables
  vif_df$Variable_fr <- sapply(vif_df$Variable, function(v) {
    if (v %in% names(labels)) {
      labels[[v]]
    } else {
      v
    }
  })

  ggplot(vif_df, aes(
    y = reorder(Variable_fr, VIF_ajuste),
    x = VIF_ajuste
  )) +
    geom_col() +
    geom_vline(xintercept = 5, linetype = "dashed") +
    labs(
      title = "Multicollinéarité des variables",
      x = "VIF ajusté",
      y = "Variables"
    ) +
    coord_cartesian(xlim = c(0, 10)) +
    theme_minimal(base_size = 14)
}
```

Enfin, la fonction `logit_graph` permet de représenter graphiquement les résultats du modèle de régression logistique. Elle extrait d'abord les coefficients du modèle à l'aide de la fonction `tidy` du package `broom`, puis retire l'intercepte afin de conserver uniquement les variables explicatives. La fonction calcule ensuite le pseudo- R^2 de McFadden, qui donne une indication globale de l'ajustement du modèle. Les coefficients sont aussi transformés en odds ratios, ce qui facilite leur interprétation. Un odds ratio supérieur à 1 indique une augmentation des

chances associées à la classe étudiée, tandis qu'un odds ratio inférieur à 1 indique une diminution de ces chances. Finalement, elle produit un graphique où les variables sont classées selon leur niveau de significativité, représenté par la valeur $-\log_{10}(\text{p-value})$. Les variables significatives au seuil de 5 % sont distinguées par une couleur différente, ce qui facilite l'identification des variables les plus importantes dans le modèle logistique. Le code de cette fonction est le suivant.

```
logit_graph <- function(modele, labels) {

  library(broom)
  library(ggplot2)
  library(dplyr)
  library(pscl)

  coef_data <- broom::tidy(modele) %>%
    filter(term != "(Intercept)")

  pseudo_r2 <- pscl::pR2(modele)["McFadden"]

  coef_data <- coef_data %>%
    mutate(
      odds_ratio = exp(estimate),
      signif = case_when(
        p.value < 0.001 ~ "****",
        p.value < 0.01  ~ "***",
        p.value < 0.05  ~ "**",
        p.value < 0.1   ~ ".",
        TRUE ~ ""
      ),
      term_fr = sapply(term, function(v) {
        if (v %in% names(labels)) {
          labels[[v]]
        } else {
          v
        }
      })
    )

  ggplot(coef_data, aes(
    x = reorder(term_fr, -p.value),
    y = -log10(p.value),
    fill = p.value < 0.05
  )) +
    geom_col(show.legend = FALSE) +
    coord_flip() +
    scale_fill_manual(values = c("TRUE" = "#1b9e77", "FALSE" = "#d95f02")) +
    geom_text(
      aes(label = paste0(signif, " OR=", round(odds_ratio, 2))),
      hjust = -0.1,
      size = 4,
      color = "black"
    ) +
    labs(
      title = "Résultats du modèle logistique",
      subtitle = paste0("Pseudo R² de McFadden = ", round(pseudo_r2, 3)),
      x = "Variables explicatives",
      y = expression(-log[10](p-value))
    ) +
}
```

```
theme_minimal(base_size = 14)
}
```

3.4.2 Modélisation par arbre de décision

Le modèle d'arbre de décision a été construit à l'aide de la fonction `rpart` avec l'argument `method = "class"`, indiquant qu'il s'agit d'un problème de classification. Ce modèle sépare les données en différentes sous-parties en fonction des variables explicatives, en créant des règles de décision successives. Il offre une représentation visuelle simple à interpréter sous forme de structure en arbre. Le code suivant présente l'implémentation du modèle.

```
modele_tree <- rpart(
  Success ~ Sleep_Hours + Physical_Activity + Hours_Studied +
    Attendance + Previous_Scores +
    Motivation_Level + Gender + Access_to_Resources +
    Parental_Education_Level + Parental_Involvement,
  data = data_train,
  method = "class"
)
```

La fonction `plot_tree_structure` permet de visualiser la structure d'un arbre de décision afin d'interpréter les règles de classification générées par le modèle. Elle utilise `rpart.plot` pour représenter les divisions successives de l'arbre, ainsi que les classes prédites et les probabilités associées. Les paramètres choisis améliorent la lisibilité du graphique en structurant l'arbre et en ajustant l'affichage des informations.

```
plot_tree_structure <- function(modele) {

  library(rpart.plot)

  rpart.plot(
    modele,
    type = 2,
    extra = 104,
    fallen.leaves = TRUE,
    cex = 0.8
  )
}
```

La fonction `tree_graph` permet de représenter graphiquement l'importance des variables explicatives dans un modèle d'arbre de décision. Elle utilise l'objet `variable.importance` contenu dans le modèle afin d'identifier les variables ayant le plus contribué aux divisions de l'arbre. La fonction commence par vérifier que le modèle contient bien des valeurs d'importance; dans le cas contraire, un message d'erreur est affiché. Les valeurs sont ensuite placées dans un tableau, puis classées en ordre décroissant afin de faire ressortir les variables les plus influentes. Les noms des variables sont aussi traduits à l'aide de la liste `labels`, ce qui rend le graphique plus clair et plus facile à interpréter. Finalement, la fonction produit un graphique à barres horizontales présentant l'importance de chaque variable dans le modèle d'arbre de décision.

```
tree_graph <- function(modele, labels, var_name = NULL) {

  library(ggplot2)
  library(dplyr)
  library(shiny)

  validate(
    need(!is.null(modele$variable.importance),
         "Aucune importance de variable disponible pour ce modèle.")
  )

  imp_data <- data.frame(
    variable = names(modele$variable.importance),
    Importance = as.numeric(modele$variable.importance)
  ) %>%
    arrange(desc(Importance))

  # Traduction des variables
  imp_data$Variable_fr <- sapply(imp_data$variable, function(v) {
    if (v %in% names(labels)) {
      labels[[v]]
    } else {
      v
    }
  })

  titre_modele <- ifelse(
    is.null(var_name),
    "Importance des variables - Arbre de décision",
    paste("Importance des variables pour", var_name)
  )

  ggplot(imp_data, aes(
    x = reorder(Variable_fr, Importance),
    y = Importance,
    fill = Importance
  )) +
    geom_col(show.legend = FALSE) +
    coord_flip() +
    scale_fill_gradient(low = "#d95f02", high = "#1b9e77") +
    labs(
      title = titre_modele,
      x = "Variables explicatives",
      y = "Importance"
    ) +
    theme_minimal(base_size = 14) +
    theme(plot.title = element_text(face = "bold"))
}
```

3.4.3 Modélisation par forêt aléatoire

Le modèle Random Forest a été construit à l'aide de la fonction `randomForest`, qui repose sur un ensemble d'arbres de décision. L'argument `importance = TRUE` permet de calculer l'importance des variables, ce qui facilite l'interprétation du modèle. Le code suivant présente l'implémentation du modèle.

```
modele_rf <- randomForest(
  Success ~ Sleep_Hours + Physical_Activity + Hours_Studied +
```

```
Attendance + Previous_Scores +  
Motivation_Level + Gender + Access_to_Resources +  
Parental_Education_Level + Parental_Involvement,  
data = data_train,  
importance = TRUE  
)
```

La fonction `rf_graph` permet de visualiser l'importance des variables dans un modèle de forêt aléatoire ainsi que l'erreur OOB. Elle vérifie que ces informations sont présentes dans le modèle, puis extrait l'erreur finale et les mesures d'importance des variables (MeanDecreaseGini ou MeanDecreaseAccuracy). Les variables sont ensuite classées et traduites à l'aide des labels pour faciliter la lecture. Enfin, un graphique à barres est généré pour présenter les variables les plus importantes.

```
rf_graph <- function(modele, labels, var_name = NULL) {  
  
  library(ggplot2)  
  library(dplyr)  
  library(randomForest)  
  library(shiny)  
  
  validate(  
    need(!is.null(modele$err.rate),  
      "Erreur OOB non disponible pour ce modèle."),  
    need(!is.null(importance(modele)),  
      "Importance des variables non disponible.")  
  )  
  
  oob_error <- tail(modele$err.rate[, "OOB"], 1)  
  
  imp_data <- as.data.frame(importance(modele))  
  imp_data$variable <- rownames(imp_data)  
  
  if ("MeanDecreaseGini" %in% colnames(imp_data)) {  
    imp_data$Importance <- imp_data$MeanDecreaseGini  
  } else if ("MeanDecreaseAccuracy" %in% colnames(imp_data)) {  
    imp_data$Importance <- imp_data$MeanDecreaseAccuracy  
  } else {  
    validate("Aucune mesure d'importance reconnue.")  
  }  
  
  imp_data <- imp_data %>% arrange(desc(Importance))  
  
  # Traduction des variables  
  imp_data$Variable_fr <- sapply(imp_data$variable, function(v) {  
    if (v %in% names(labels)) {  
      labels[[v]]  
    } else {  
      v  
    }  
  })  
  
  titre_modele <- ifelse(  
    is.null(var_name),  
    "Importance des variables - Random Forest",  
    paste("Importance des variables pour", var_name)  
  )  
}
```



```
ggplot(imp_data, aes(
  x = reorder(Variable_fr, Importance),
  y = Importance,
  fill = Importance
)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  scale_fill_gradient(low = "#d95f02", high = "#1b9e77") +
  labs(
    title = titre_modele,
    subtitle = paste0("Erreur OOB = ", round(oob_error, 3)),
    x = "Variables explicatives",
    y = "Importance"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 12, color = "gray30")
  )
}
```

3.5 Évaluation et comparaison des modèles

Une fois les modèles construits, leur performance a été évaluée et comparée sur le jeu de test afin de mesurer leur capacité à prédire de nouvelles observations.

La fonction `evaluer_modeles` permet d'évaluer et de comparer plusieurs modèles de classification à partir des données de test. Elle reçoit une liste de modèles, l'ensemble `data_test`, la variable cible ainsi que la classe considérée comme positive, ici *Failure*. Pour la régression logistique, les probabilités prédites sont transformées en classes à partir d'un seuil de 0,5. Pour les autres modèles, comme l'arbre de décision et la forêt aléatoire, les classes sont prédites directement. Les prédictions sont ensuite comparées aux vraies valeurs de la variable cible à l'aide d'une matrice de confusion. À partir de cette matrice, la fonction extrait l'accuracy, la précision et le rappel. Le score F1 est ensuite calculé à partir de la précision et du rappel afin d'obtenir une mesure qui tient compte de ces deux indicateurs. Finalement, un graphique à barres est généré afin de comparer les modèles selon chaque métrique.

```
evaluer_modeles <- function(models, data_test, variable_cible = "Success",
                             positive_class = "Failure") {

  library(ggplot2)
  library(dplyr)
  library(tidyr)
  library(caret)

  validate(
    need(length(models) > 0, "Aucun modèle fourni.")
  )

  evaluer <- function(modele, data_test, variable_cible, nom_modele,
    positive_class) {

    if ("glm" %in% class(modele)) {
```

```
    proba <- predict(modele, newdata = data_test, type = "response")
    pred <- ifelse(proba >= 0.5, "Success", "Failure")
  } else {
    pred <- predict(modele, newdata = data_test, type = "class")
  }

  pred <- factor(pred, levels = levels(data_test[[variable_cible]]))

  cm <- confusionMatrix(pred, data_test[[variable_cible]], positive =
positive_class)

  accuracy <- as.numeric(cm$overall["Accuracy"])
  precision <- as.numeric(cm$byClass["Pos Pred Value"])
  recall <- as.numeric(cm$byClass["Sensitivity"])

  f1 <- ifelse((precision + recall) == 0,
              NA,
              2 * (precision * recall) / (precision + recall))

  data.frame(
    Modele = nom_modele,
    Accuracy = accuracy,
    Precision = precision,
    Recall = recall,
    F1 = f1
  )
}

res <- do.call(rbind, lapply(models, function(m) {
  evaluer(m$modele, data_test, variable_cible, m$nom, positive_class)
}))

res_long <- res %>%
  pivot_longer(cols = c("Accuracy", "Precision", "Recall", "F1"),
               names_to = "Metrique",
               values_to = "Valeur")

ggplot(res_long, aes(x = Metrique, y = Valeur, fill = Modele)) +
  geom_col(position = position_dodge(0.9)) +
  geom_text(
    aes(label = round(Valeur, 3)),
    position = position_dodge(0.9),
    vjust = -0.3,
    size = 3.5
  ) +
  labs(
    title = "Comparaison des modèles",
    x = "Métrique",
    y = "Valeur"
  ) +
  ylim(0, 1.1) +
  theme_minimal()
}
```

3.6 Analyse complémentaire

Cette section présente les fonctions utilisées l'analyse complémentaire des résultats. Elles permettent d'examiner les profils d'étudiants associés aux résultats extrêmes, soit les résultats très élevés et les résultats faibles.

La fonction `extreme_plot_quali` permet de comparer les variables qualitatives selon deux groupes d'étudiants : ceux ayant obtenu un résultat élevé à l'examen, soit un score de 95 ou plus, et ceux ayant obtenu un résultat faible, soit un score inférieur à 60. La fonction commence par créer une nouvelle variable appelée `Groupe`, qui classe les observations selon ces deux catégories. Les étudiants ayant un résultat entre 60 et 94 sont retirés de cette analyse afin de se concentrer uniquement sur les profils extrêmes.

Ensuite, pour chaque variable qualitative sélectionnée, la fonction génère un graphique à barres proportionnelles. Ce type de graphique permet de comparer la répartition des modalités entre les deux groupes. Les noms des variables et des modalités sont aussi traduits à l'aide des objets `labels` et `labels_levels`, ce qui rend les graphiques plus lisibles.

```
extreme_plot_quali <- function(data, vars, labels, labels_levels,
                               score_var = "Exam_Score", ncol = 2) {

  library(ggplot2)
  library(dplyr)
  library(patchwork)
  library(scales)
  library(shiny)

  validate(
    need(length(vars) > 0, "Aucune variable sélectionnée.")
  )

  data_extreme <- data %>%
    mutate(
      Groupe = case_when(
        .data[[score_var]] >= 95 ~ "Élevé (≥95)",
        .data[[score_var]] < 60 ~ "Faible (<60)",
        TRUE ~ NA_character_
      )
    ) %>%
    filter(!is.na(Groupe))

  plots <- lapply(vars, function(v) {

    var_label <- ifelse(v %in% names(labels), labels[[v]], v)

    ggplot(data_extreme, aes(x = .data[[v]], fill = Groupe)) +
      geom_bar(position = "fill") +
      scale_x_discrete(labels = labels_levels) +
      scale_y_continuous(labels = percent) +
      labs(
        title = paste("Résultats extrêmes selon", var_label),
        x = var_label,
        y = "Proportion"
      ) +
      theme_minimal()
  })

  patchwork::wrap_plots(plots, ncol = ncol)
}
```

La fonction `extreme_plot_num` permet d'analyser les variables quantitatives selon les mêmes groupes de résultats extrêmes.

Pour chaque variable numérique sélectionnée, la fonction produit un diagramme en boîte. Ce type de graphique permet de comparer la distribution des valeurs entre les deux groupes. Il permet notamment d'observer les différences de médiane, de dispersion et de valeurs extrêmes. Les noms des variables sont également traduits à l'aide de l'objet `labels`, ce qui améliore la clarté de la présentation graphique.

```
extreme_plot_num <- function(data, vars, labels, score_var = "Exam_Score", ncol =
2) {

  library(ggplot2)
  library(dplyr)
  library(patchwork)
  library(shiny)

  validate(
    need(length(vars) > 0, "Aucune variable sélectionnée.")
  )

  data_extreme <- data %>%
    mutate(
      Groupe = case_when(
        .data[[score_var]] >= 95 ~ "Élevé (>95)",
        .data[[score_var]] < 60 ~ "Faible (<60)",
        TRUE ~ NA_character_
      )
    ) %>%
    filter(!is.na(Groupe))

  plots <- lapply(vars, function(v) {

    var_label <- ifelse(v %in% names(labels), labels[[v]], v)

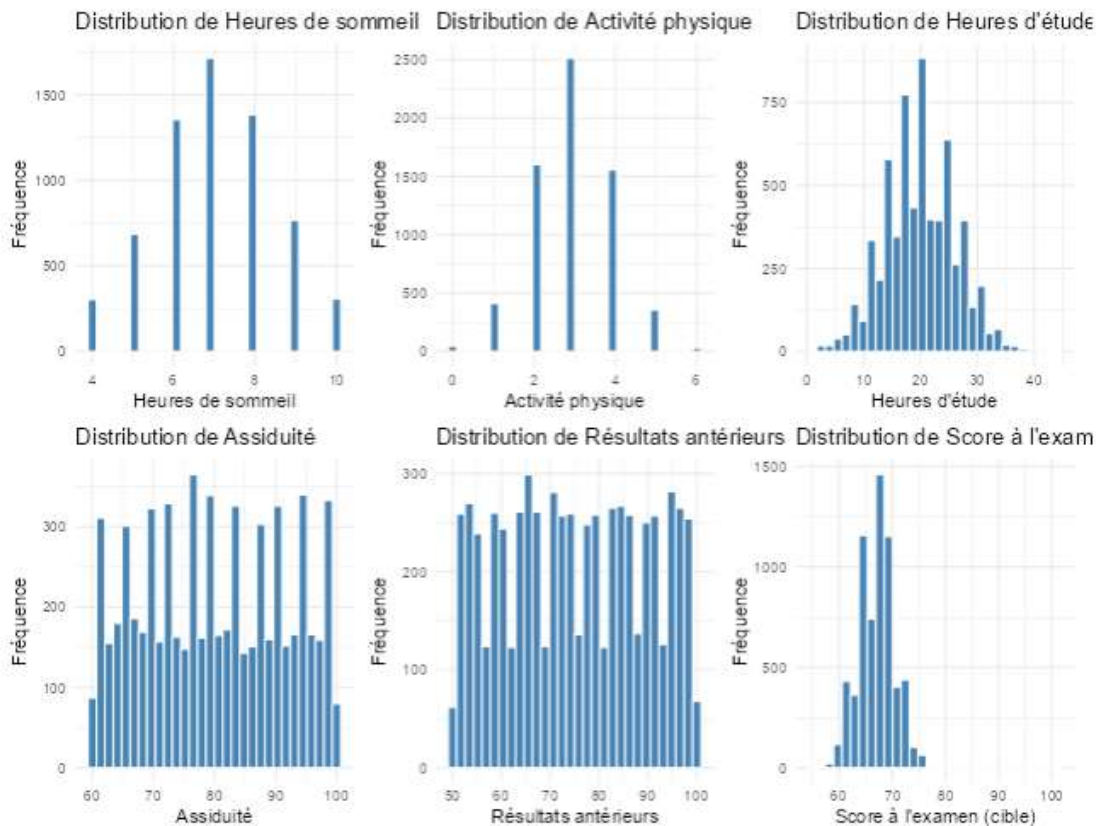
    ggplot(data_extreme, aes(x = Groupe, y = .data[[v]], fill = Groupe)) +
      geom_boxplot() +
      labs(
        title = paste("Résultats extrêmes selon", var_label),
        x = "Groupe",
        y = var_label
      ) +
      theme_minimal() +
      theme(legend.position = "none")
  })

  patchwork::wrap_plots(plots, ncol = ncol)
}
```

4. Résultats

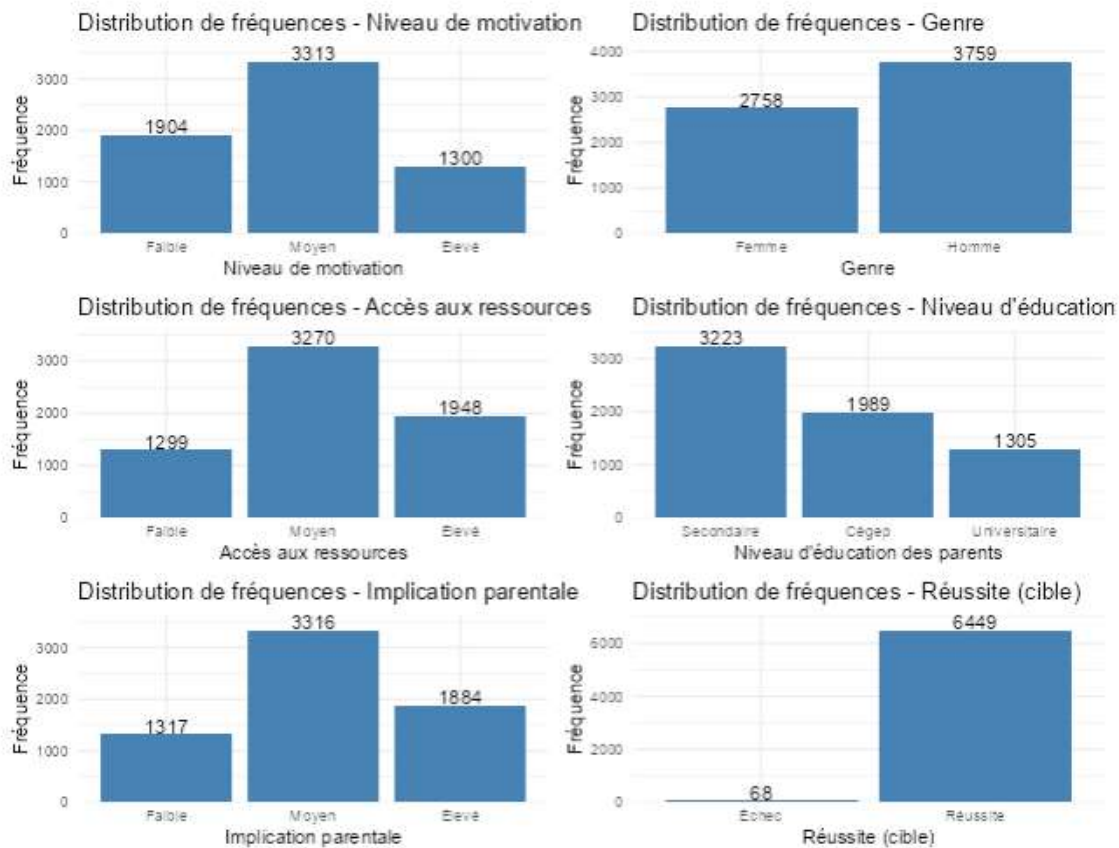
Cette section présente les résultats obtenus à partir des analyses exploratoires et des modèles de classification. Les graphiques générés sont utilisés afin de mettre en évidence les principales tendances des données, d'évaluer la performance des modèles et d'approfondir l'interprétation des relations entre les variables.

Figure 1 : Résultats de la distribution des variables quantitatives



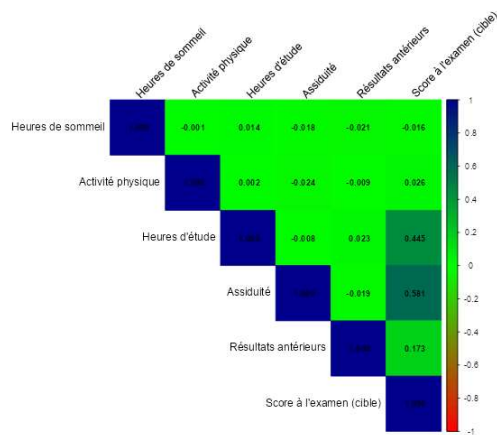
Les distributions des variables numériques révèlent des profils distincts. Les heures de sommeil sont majoritairement comprises entre 6 et 8 heures, indiquant un comportement relativement stable. L'activité physique se concentre autour de niveaux modérés, principalement entre 2 et 4. Les heures d'étude présentent une distribution en forme de cloche centrée autour d'environ 20 heures, suggérant une répartition plus structurée. L'assiduité et les résultats antérieurs sont répartis de manière assez uniforme sur leur intervalle, ce qui indique une bonne diversité des profils d'étudiants. Enfin, le score à l'examen, qui constitue la variable cible, est principalement concentré entre 65 et 75, montrant que la majorité des étudiants obtient des résultats moyens.

Figure 2 : Résultats de la distribution des variables qualitatives



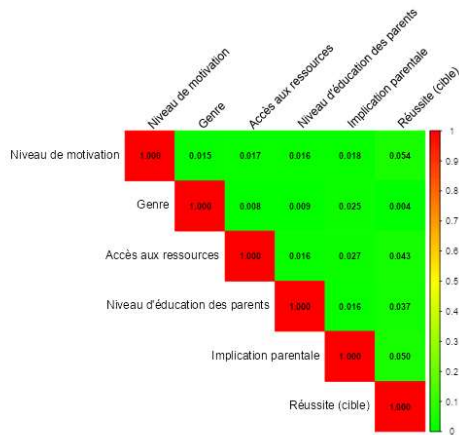
Les distributions des variables qualitatives montrent certaines tendances dans les données. Le niveau de motivation est principalement concentré au niveau moyen. Le genre présente une légère majorité d'hommes. L'accès aux ressources est majoritairement de niveau moyen. Le niveau d'éducation des parents est principalement de niveau secondaire, suivi du cégep, puis universitaire. L'implication parentale est également dominée par un niveau moyen. Enfin, la variable cible *Success* est fortement déséquilibrée, avec une très grande majorité d'étudiants en situation de réussite et très peu en échec.

Figure 3 : Résultats de la corrélation des variables quantitatives



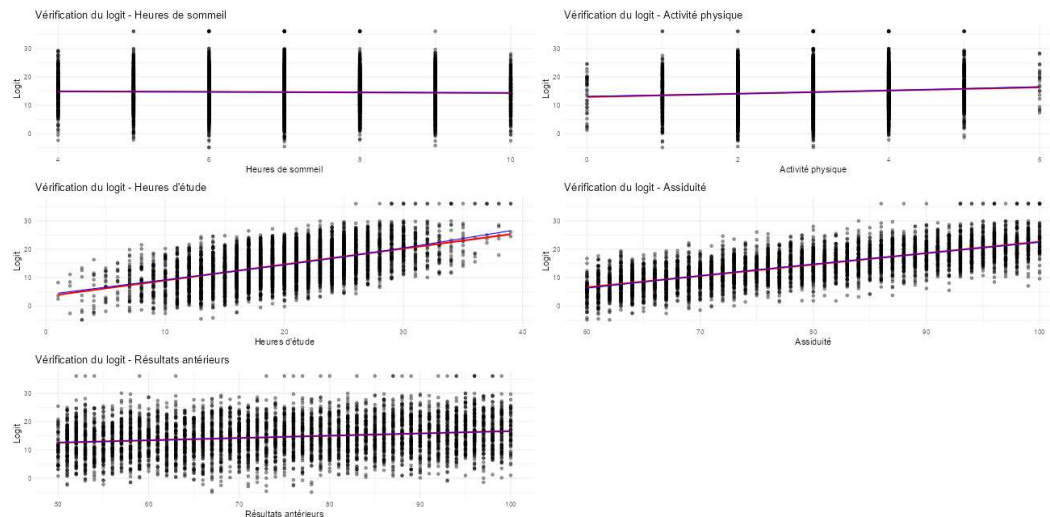
Les corrélations entre les variables numériques montrent des relations faibles à modérées. La majorité des variables, telles que les heures de sommeil, l'activité physique et l'assiduité, présentent des corrélations très faibles entre elles, proches de zéro, indiquant une faible dépendance linéaire. En revanche, certaines relations plus marquées apparaissent avec la variable cible. Le score à l'examen est modérément corrélé avec les heures d'étude (≈ 0.45) et plus fortement avec l'assiduité (≈ 0.58), suggérant que ces variables jouent un rôle dans la réussite académique.

Figure 4 : Résultats de la corrélation des variables qualitatives



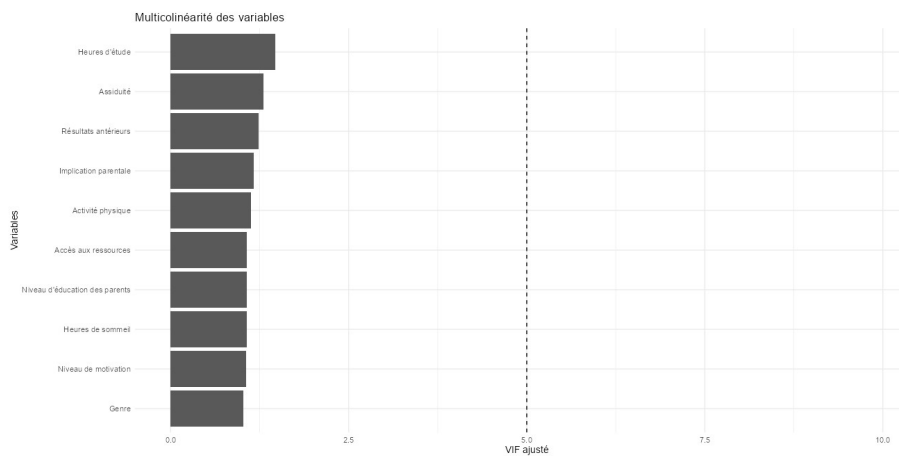
Les corrélations entre les variables qualitatives, mesurées à l'aide du coefficient de Cramér's V, sont globalement très faibles. La majorité des valeurs se situent près de zéro, ce qui indique une faible association entre les variables. De même, les relations entre ces variables et la variable cible *Success* sont également faibles, suggérant qu'aucune variable qualitative ne présente à elle seule une forte association avec la réussite.

Figure 5 : Résultats de la linéarité du logit du modèle de régression logistique



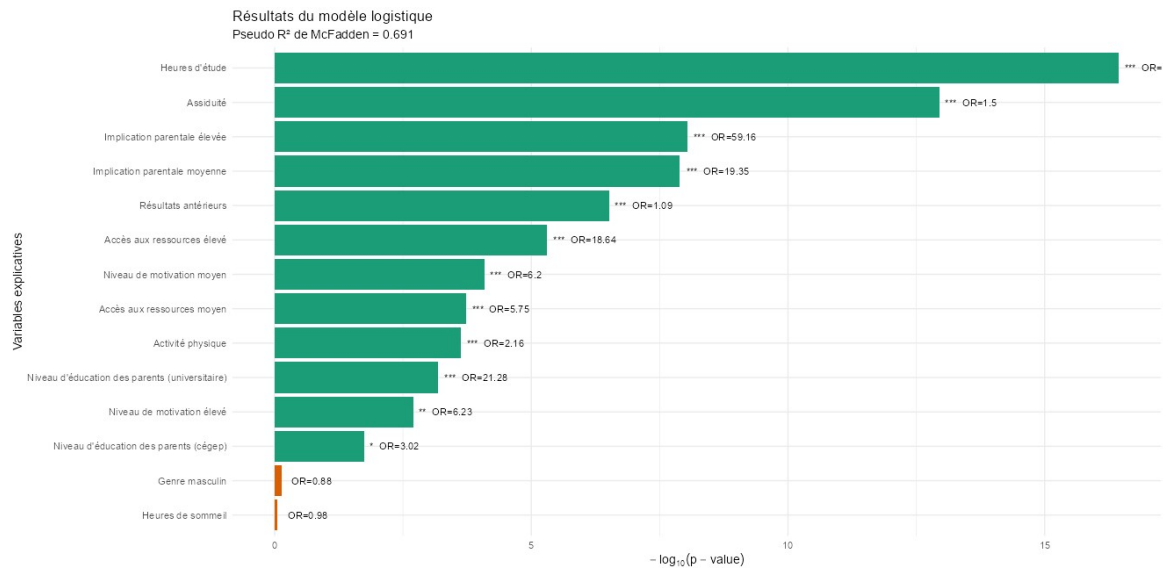
Les résultats montrent que, pour l'ensemble des variables, la courbe lissée suit étroitement la droite de régression linéaire, indiquant une relation globalement linéaire entre le logit et les variables. Cela suggère que l'hypothèse de linéarité du logit est respectée. Par conséquent, cette vérification renforce la validité du modèle de régression logistique et la fiabilité des résultats obtenus.

Figure 6 : Résultats de la multicollinéarité des variables du modèle de régression logistique



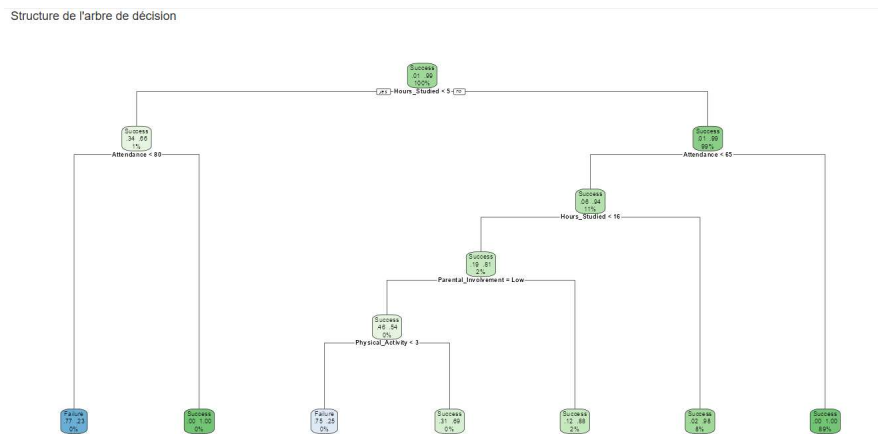
Les valeurs de VIF pour l'ensemble des variables sont faibles (toutes bien en dessous du seuil critique de 5), indiquant une absence de multicollinéarité problématique. Les variables explicatives ne présentent donc pas de forte redondance entre elles, ce qui assure la stabilité et la fiabilité des coefficients du modèle de régression logistique.

Figure 7 : Résultats du modèle de régression logistique



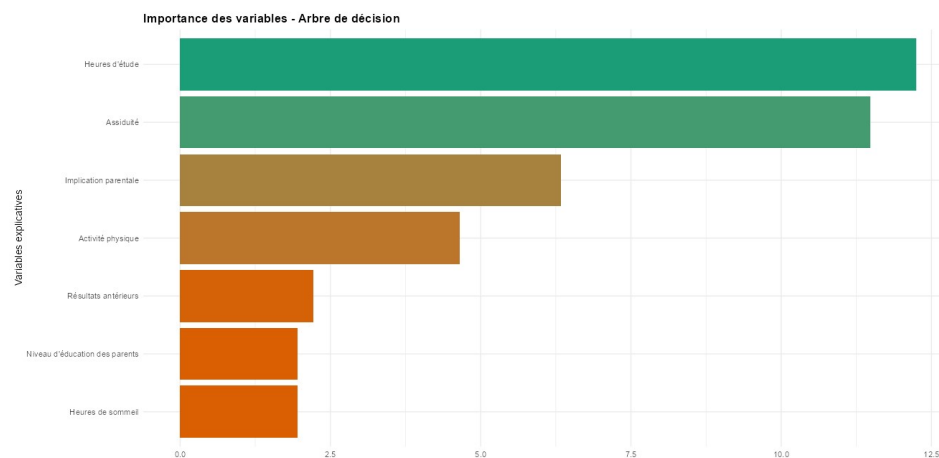
Les variables avec les barres les plus longues (donc p-value très faible) sont les plus influentes, notamment les heures d'étude, l'assiduité, l'implication parentale (élevée et moyenne), les résultats antérieurs, ainsi que l'accès aux ressources et le niveau de motivation. Ces variables présentent un impact significatif sur la probabilité de réussite, et leurs odds ratios supérieurs à 1 indiquent qu'une augmentation de ces facteurs est associée à une hausse des chances de réussite. Par exemple, les heures d'étude et l'implication parentale élevée ont un effet particulièrement marqué, tout comme l'accès aux ressources et la motivation. À l'inverse, des variables comme le genre masculin et les heures de sommeil affichent des odds ratios proches de 1 et une faible significativité, ce qui suggère qu'elles ont peu ou pas d'influence dans ce modèle. Dans l'ensemble, ces résultats montrent que la réussite scolaire dépend principalement de l'engagement académique de l'élève (heures d'étude, assiduité), de son environnement familial et des ressources éducatives disponibles. Enfin, la valeur du pseudo R² de McFadden (0,691) indique que le modèle possède une très bonne capacité explicative, (0.6 et plus est considéré comme très élevé).

Figure 8 : Résultats du modèle d'arbre de décision (structure en arbre)



L'arbre de décision montre que la variable la plus importante pour prédire la réussite est le nombre d'heures d'étude, puisqu'elle constitue la racine de l'arbre. Les élèves qui étudient moins de 5 heures présentent un risque élevé d'échec, surtout lorsque leur assiduité est inférieure à 80 %, ce qui confirme que le manque d'étude combiné à une faible présence en classe est fortement associé à l'échec. À l'inverse, les élèves qui étudient au moins 5 heures ont une probabilité de réussite très élevée, particulièrement si leur assiduité dépasse 65 %, ce qui représente le scénario le plus fréquent. Dans les cas intermédiaires, une faible implication parentale peut être compensée en partie par un niveau suffisant d'activité physique, tandis qu'une combinaison défavorable augmente le risque d'échec. Dans l'ensemble, cet arbre met en évidence que la réussite scolaire repose principalement sur l'engagement de l'élève (heures d'étude et assiduité), tandis que les autres facteurs jouent un rôle complémentaire.

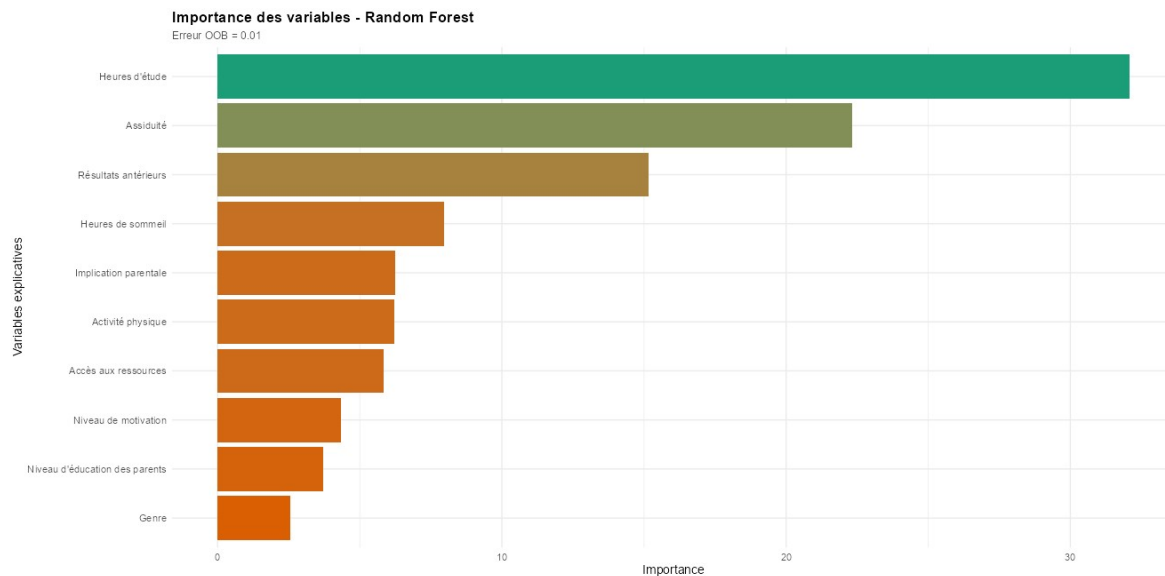
Figure 9 : Résultats du modèle d'arbre de décision (diagramme)



Ce graphique de l'importance des variables dans l'arbre de décision apporte une visualisation complémentaire des facteurs influençant la réussite, en confirmant les résultats observés

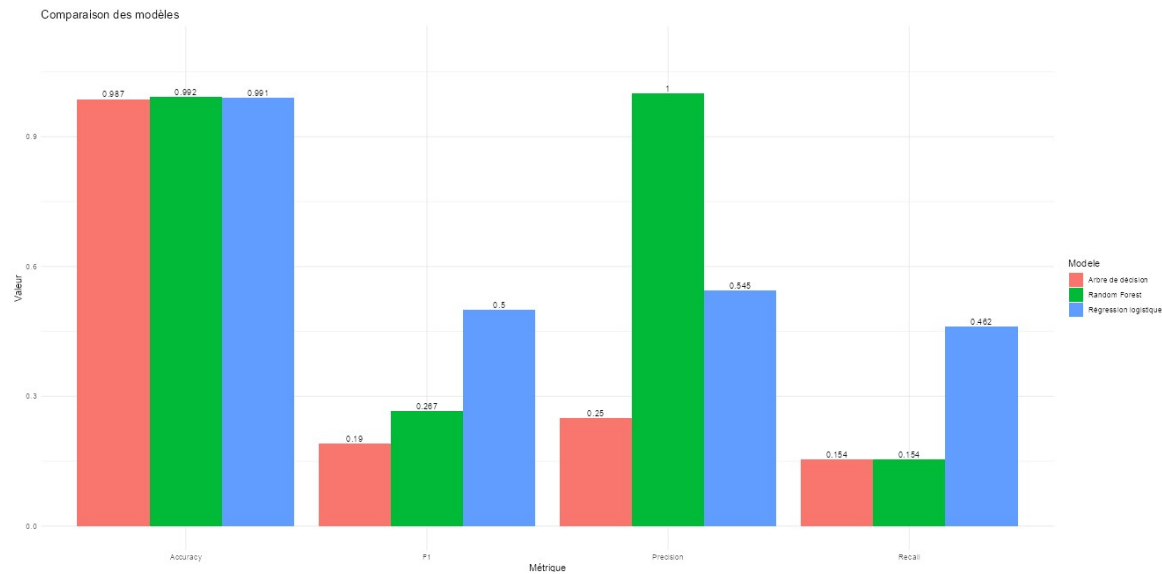
précédemment. On constate que les heures d'étude et l'assiduité sont les variables les plus importantes. Ensuite, l'implication parentale et l'activité physique présentent une contribution secondaire. À l'inverse, les résultats antérieurs, le niveau d'éducation des parents et les heures de sommeil ont une importance plus faible dans ce modèle.

Figure 10 : Résultats du modèle de forêt aléatoire



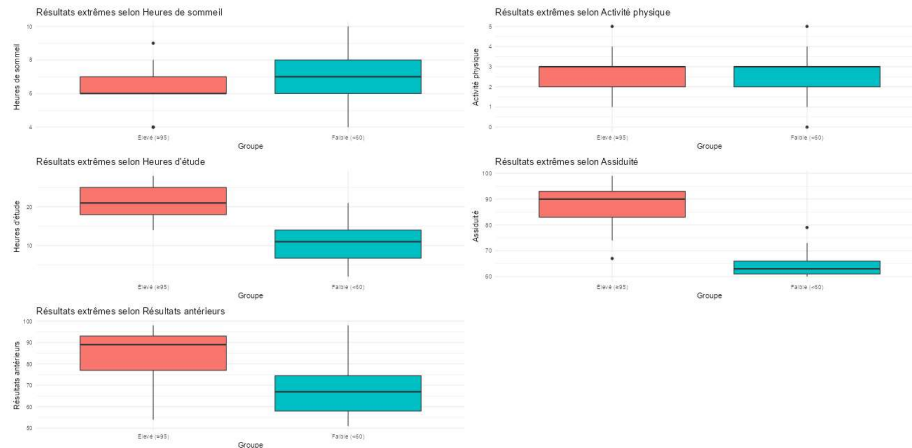
Ce graphique de l'importance des variables dans le modèle de forêt aléatoire montre que les heures d'étude et l'assiduité sont les facteurs les plus déterminants dans la prédiction de la réussite. Les résultats antérieurs occupent également une place importante, indiquant que les performances passées contribuent significativement à la prédiction. Ensuite, des variables comme les heures de sommeil, l'implication parentale, l'activité physique et l'accès aux ressources présentent une importance modérée. Le niveau de motivation et le niveau d'éducation des parents ont un impact plus faible, tandis que le genre apparaît comme la variable la moins importante dans ce modèle. Par ailleurs, l'erreur OOB très faible (0,01) indique que le modèle possède une excellente capacité de généralisation.

Figure 11 : Résultats réels selon les métriques pour la comparaison des trois modèles



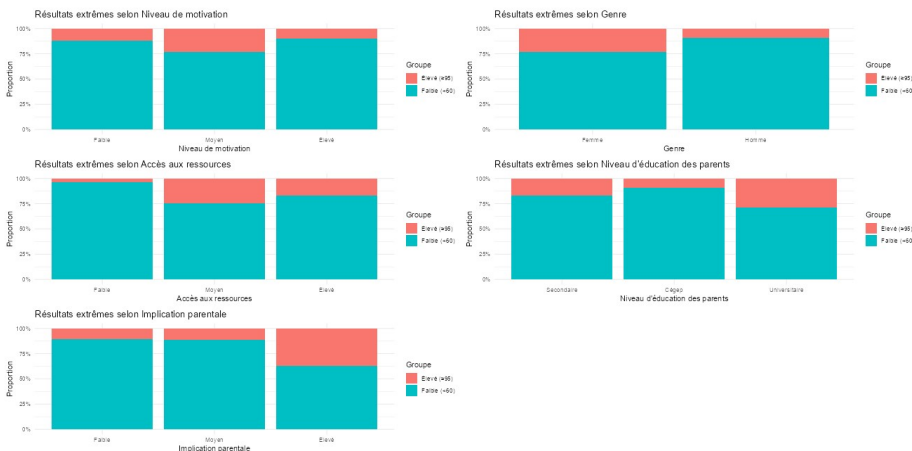
Ce graphique de comparaison des modèles montre que les trois approches (arbre de décision, forêt aléatoire et régression logistique) obtiennent une accuracy très élevée, toutes supérieures à 0,98 (un bon modèle dépasse généralement 0,8), ce qui indique qu'elles classent très bien les observations. Cependant, des différences importantes apparaissent au niveau des autres métriques. Le recall, qui mesure la capacité à détecter les cas d'échec, est faible pour l'arbre (0,154) et la forêt (0,154), ce qui est considéré comme insuffisant (un bon recall se situe généralement au-dessus de 0,7), tandis que la régression logistique atteint 0,462, ce qui demeure modéré, mais nettement supérieur aux autres modèles. En termes de précision, la forêt aléatoire atteint 1, soit une précision parfaite, indiquant qu'elle ne fait pratiquement pas d'erreurs lorsqu'elle prédit un cas positif, mais son recall plus faible montre qu'elle en détecte peu. Le F1-score, qui équilibre précision et rappel, est également faible pour l'arbre (0,19) et la forêt (0,267), alors que la régression logistique atteint 0,5, ce qui correspond à une performance moyenne. En somme, malgré des accuracy similaires, la régression logistique offre le meilleur compromis pour prédire les cas d'échec scolaire dans notre jeu de données.

Figure 12 : Résultats de l'analyse complémentaire des extrêmes pour les variables quantitatives



Les élèves ayant des notes élevées semblent avoir légèrement moins d'heures de sommeil. L'activité physique n'a pas d'impact clair, puisqu'elle ne favorise pas une classe plus qu'une autre. En revanche, une différence très marquée est observée pour les heures d'étude, l'assiduité et les résultats antérieurs, qui distinguent fortement les élèves en réussite extrême de ceux en échec.

Figure 13 : Résultats de l'analyse complémentaire des extrêmes pour les variables qualitatives



La motivation moyenne reflète la plus grande proportion de cas extrêmes. Le genre montre une légère différence en faveur du sexe féminin. Un faible accès aux ressources contient la plus grande proportion de cas d'échec. Finalement, un niveau d'éducation parental universitaire ainsi qu'une implication parentale élevée sont associés à moins d'échecs et à une plus grande proportion de réussites extrêmes.

5. Discussion

Contrairement à notre hypothèse H1, le modèle de forêt aléatoire n'a pas offert la meilleure performance prédictive. C'est plutôt la régression logistique qui s'est démarquée dans notre cas. Ce résultat peut s'expliquer par le fait que les hypothèses du modèle logistique étaient très bien respectées, notamment une linéarité presque parfaite du logit. Cela montre que ce type de modèle peut être suffisant lorsque ses conditions d'application sont remplies. De plus, les données étant fortement déséquilibrées, avec seulement environ 1 % de cas d'échec, cela peut avoir influencé les performances des modèles (James et al., 2021). Ainsi, l'hypothèse H1 est infirmée, puisque la forêt aléatoire n'a pas offert la meilleure performance prédictive.

La comparaison de l'importance des variables entre les trois modèles (régression logistique, arbre de décision et forêt aléatoire) montre des résultats assez similaires. Les heures d'étude arrivent en premier dans les trois modèles, suivies de l'assiduité, ce qui confirme que l'implication des élèves joue un rôle central dans leur réussite scolaire. La troisième variable d'importance pour la régression logistique et pour l'arbre de décision est l'implication des parents, tandis que pour la forêt aléatoire, ce sont les résultats antérieurs. Les autres variables, comme l'accès aux ressources, la motivation, l'activité physique et le niveau d'étude des parents, ont aussi une certaine importance, mais moins marquée. À l'inverse, des variables comme le genre ou les heures de sommeil ont peu d'impact. Ainsi, ces résultats montrent que la réussite dépend principalement de l'engagement de l'élève.

Dans le but de répondre à l'hypothèse H2, la notion d'environnement scolaire n'a pas été définie comme une catégorie distincte dans les variables initiales, mais plutôt comme un concept englobant certains facteurs liés au contexte d'apprentissage de l'élève. Plus précisément, cet environnement inclut la variable académique, soit les résultats antérieurs, ainsi que certaines variables reflétant les conditions d'apprentissage, comme l'accès aux ressources et l'implication parentale. Les résultats montrent que, bien que les variables comportementales liées à l'engagement de l'élève soient les plus déterminantes dans la réussite scolaire, les variables associées à l'environnement scolaire demeurent tout de même plus influentes que les variables socio-démographiques, soit le genre et le niveau d'étude des parents. Ainsi, l'hypothèse H2, selon laquelle les variables liées à l'environnement scolaire ont plus d'impact que les variables socio-démographiques, est confirmée.

Finalement, l'analyse des résultats extrêmes permet d'identifier certains profils d'étudiants associés à la réussite ou à l'échec. D'un côté, les élèves en réussite extrême (≥ 95) se distinguent par un niveau d'engagement élevé, soit davantage d'heures d'étude, une assiduité plus forte et de meilleurs résultats antérieurs. Ils sont également associés à une implication parentale élevée ou à des parents de niveau universitaire. À l'inverse, les élèves en situation d'échec (< 60) présentent moins d'heures d'étude, une assiduité plus faible et des résultats antérieurs plus bas. Ils sont aussi associés à un accès limité aux ressources. Cependant, le fait d'avoir une implication parentale faible ou des parents de niveau secondaire n'a pas d'impact ici. Ainsi, les résultats observés dans les graphiques appuient l'hypothèse H3, puisque les performances extrêmes sont bien associées à des profils spécifiques d'étudiants.

6. Conclusion

En conclusion, cette étude a permis d'analyser les facteurs influençant la réussite scolaire à l'aide de différentes méthodes de modélisation. Les résultats montrent que la régression logistique est le modèle le plus performant dans ce contexte, notamment en raison de sa meilleure capacité à détecter les cas d'échec, malgré un déséquilibre important dans les données. L'analyse met en évidence que l'engagement de l'étudiant, notamment à travers les heures d'étude et l'assiduité, constitue le facteur le plus déterminant de la réussite.

Pour aller plus loin, il serait pertinent de reproduire cette analyse à partir d'autres jeux de données, possiblement plus diversifiés ou mieux équilibrés, afin de valider la robustesse des résultats obtenus. Cela permettrait également de vérifier si les facteurs identifiés demeurent constants dans différents contextes éducatifs et d'améliorer la généralisation des conclusions.

7. Bibliographie

James, G., Tibshirani, R., Hastie, T., & Witten, D. (2021). *Introduction to Statistical Learning : With Applications in R*. Springer.

Kaggle. (s. d.). *Student exam performance dataset analysis*. Consulté le 13 mars 2026.
<https://www.kaggle.com/datasets/grandmaster07/student-exam-performance-dataset-analysis>

8. Annexe

```
#-----  
# 4. Fonction Shiny pour la visualisation et l'analyse des données et des modèles  
#-----  
dashboard <- function(data, data_test, data_train, modele_logit, modele_tree,  
modele_rf) {  
  
  library(shiny)  
  library(ggplot2)  
  library(patchwork)  
  
  # Traduction des variables numériques  
  labels_num_fr <- c(  
    Sleep_Hours = "Heures de sommeil",  
    Physical_Activity = "Activité physique",  
    Hours_Studied = "Heures d'étude",  
    Attendance = "Assiduité",  
    Previous_Scores = "Résultats antérieurs",  
    Exam_Score = "Score à l'examen (cible)"  
  )  
  
  # Traduction des variables qualitatives  
  labels_quali_fr <- c(  
    Motivation_Level = "Niveau de motivation",  
    Gender = "Genre",  
    Access_to_Resources = "Accès aux ressources",
```

```

    Parental_Education_Level = "Niveau d'éducation des parents",
    Parental_Involvement = "Implication parentale",
    Success = "Réussite (cible)",
    Motivation_LevelMedium = "Niveau de motivation moyen",
    Motivation_LevelHigh = "Niveau de motivation élevé",
    GenderMale = "Genre masculin",
    Access_to_ResourcesMedium = "Accès aux ressources moyen",
    Access_to_ResourcesHigh = "Accès aux ressources élevé",
    Parental_Education_LevelCollege = "Niveau d'éducation des parents cégep",
    Parental_Education_LevelPostgraduate = "Niveau d'éducation des parents
universitaire",
    Parental_InvolvementMedium = "Implication parentale moyenne",
    Parental_InvolvementHigh = "Implication parentale élevée"
  )

# Traduction pour toutes les variables
labels_tous_fr <- c(labels_num_fr, labels_quali_fr)

# Traduction pour les classes
labels_levels_fr <- c(
  "Low" = "Faible",
  "Medium" = "Moyen",
  "High" = "Élevé",
  "Male" = "Homme",
  "Female" = "Femme",
  "Failure" = "Échec",
  "Success" = "Réussite",
  "High School" = "Secondaire",
  "College" = "Cégep",
  "Postgraduate" = "Universitaire"
)

models <- list(
  list(nom = "Régression logistique", modele = modele_logit),
  list(nom = "Arbre de décision", modele = modele_tree),
  list(nom = "Random Forest", modele = modele_rf)
)

# Interface utilisateur (UI) du dashboard Shiny
ui <- fluidPage(

  titlePanel("Dashboard réussite scolaire"),

  tabsetPanel(

    navbarMenu(
      "Exploration des données",

      tabPanel(
        "Variables numériques",

        fluidRow(
          column(
            width = 8,
            checkboxGroupInput(
              inputId = "variables_explicatives_num",
              label = "Variables numériques :",
              choices = setNames(names(labels_num_fr), labels_num_fr),
              selected = names(labels_num_fr),
              inline = TRUE
            )
          )
        )
      )
    )
  )
)

```



```
    ),
    column(
      width = 4,
      sliderInput(
        inputId = "bins",
        label = "Nombre de classes",
        min = 5,
        max = 50,
        value = 30
      )
    )
  ),
  fluidRow(
    column(
      width = 6,
      h4("Distribution"),
      plotOutput("hist_plot", height = "500px")
    ),
    column(
      width = 6,
      h4("Corrélation"),
      plotOutput("correlation_num", height = "500px")
    )
  )
),
tabPanel(
  "Variables qualitatives",

  fluidRow(
    column(
      width = 12,
      checkboxGroupInput(
        inputId = "variables_explicatives_quali",
        label = "Variables qualitatives :",
        choices = setNames(names(labels_quali_fr), labels_quali_fr),
        selected = names(labels_quali_fr),
        inline = TRUE
      )
    )
  ),
  fluidRow(
    column(
      width = 6,
      h4("Distribution"),
      plotOutput("barplot_quali", height = "500px")
    ),
    column(
      width = 6,
      h4("Association (V de Cramér)"),
      plotOutput("cramer_quali", height = "500px")
    )
  )
),
navbarMenu(
  "Modélisation",
```

```

tabPanel(
  "Régression logistique",

  h3("Régression logistique"),

  tabsetPanel(

    tabPanel(
      "Linéarité",

      fluidRow(
        column(
          width = 12,
          checkboxGroupInput(
            inputId = "variables_logit",
            label = "Variables numériques pour la vérification du logit

: ",

            choices = setNames(names(labels_num_fr), labels_num_fr),
            selected = setdiff(names(labels_num_fr), "Exam_Score"),
            inline = TRUE
          )
        )
      ),

      fluidRow(
        column(
          width = 12,
          h4("Linéarité du logit"),
          plotOutput("verif_logit_plot", height = "650px")
        )
      ),

    tabPanel(
      "VIF",
      fluidRow(
        column(
          width = 12,
          h4("Multicolinéarité (VIF)"),
          plotOutput("vif_plot", height = "650px")
        )
      ),

    tabPanel(
      "Résultats du modèle",
      fluidRow(
        column(
          width = 12,
          h4("Résultats du modèle logistique"),
          plotOutput("logit_graph", height = "650px")
        )
      ),

    tabPanel(
      "Arbre de décision",

      h3("Arbre de décision"),

```

```
tabsetPanel(  
  tabPanel("Structure de l'arbre",  
    fluidRow(  
      column(  
        width = 12,  
        h4("Structure de l'arbre de décision"),  
        plotOutput("arbre_decision_plot", height = "650px")  
      )  
    ),  
  ),  
  tabPanel("Résultats du modèle",  
    fluidRow(  
      column(  
        width = 12,  
        h4("Importance des variables"),  
        plotOutput("tree_graph", height = "650px")  
      )  
    ),  
  ),  
  tabPanel(  
    "Random Forest",  
    h3("Random Forest"),  
    fluidRow(  
      column(  
        width = 12,  
        h4("Importance des variables et erreur OOB"),  
        plotOutput("rf_graph", height = "650px")  
      )  
    ),  
  ),  
  tabPanel(  
    "Comparaison des modèles",  
    h3("Comparaison des performances réelles"),  
    fluidRow(  
      column(  
        width = 12,  
        h4("Accuracy, Precision, Recall et F1"),  
        plotOutput("eval_models", height = "650px")  
      )  
    ),  
  ),  
),  
navbarMenu(  
  "Résultats extrêmes",  
  tabPanel(  
    "Extrêmes numériques",  
    h3("Analyse des résultats extrêmes - variables numériques"),
```

```

fluidRow(
  column(
    width = 12,
    checkboxGroupInput(
      inputId = "vars_extreme_num",
      label = "Variables numériques à comparer :",
      choices = setNames(
        setdiff(names(labels_num_fr), "Exam_Score"),
        labels_num_fr[setdiff(names(labels_num_fr), "Exam_Score")]
      ),
      selected = setdiff(names(labels_num_fr), "Exam_Score"),
      inline = TRUE
    )
  )
),

fluidRow(
  column(
    width = 12,
    plotOutput("extreme_num", height = "650px")
  )
),

tabPanel(
  "Extrêmes qualitatives",

  h3("Analyse des résultats extrêmes - variables qualitatives"),

  fluidRow(
    column(
      width = 12,
      checkboxGroupInput(
        inputId = "vars_extreme_quali",
        label = "Variables qualitatives à comparer :",
        choices = setNames(
          setdiff(names(labels_quali_fr), "Success"),
          labels_quali_fr[setdiff(names(labels_quali_fr), "Success")]
        ),
        selected = setdiff(names(labels_quali_fr), "Success"),
        inline = TRUE
      )
    )
  ),

  fluidRow(
    column(
      width = 12,
      plotOutput("extreme_quali", height = "650px")
    )
  )
)
)
)
)

# Logique serveur du dashboard Shiny
server <- function(input, output) {

  output$hist_plot <- renderPlot({

```

```
    req(input$variables_explicatives_num)
  histplot(
    data = data,
    var_names = input$variables_explicatives_num,
    bins = input$bins,
    labels = labels_num_fr
  )
})

output$barplot_quali <- renderPlot({
  req(input$variables_explicatives_quali)
  barplot_quali(
    data = data,
    var_names = input$variables_explicatives_quali,
    ncol = 2,
    labels = labels_quali_fr,
    labels_levels = labels_levels_fr
  )
})

output$correlation_num <- renderPlot({
  req(input$variables_explicatives_num)
  correlation(
    data = data,
    vars = input$variables_explicatives_num,
    labels = labels_num_fr
  )
})

output$cramer_quali <- renderPlot({
  req(input$variables_explicatives_quali)
  cramer(
    data = data,
    vars = input$variables_explicatives_quali,
    labels = labels_quali_fr
  )
})

output$verif_logit_plot <- renderPlot({
  req(input$variables_logit)
  verif_logit(
    data = data_train,
    vars = input$variables_logit,
    modele = modele_logit,
    ncol = 2,
    labels = labels_num_fr
  )
})

output$logit_graph <- renderPlot({
  logit_graph(
    modele = modele_logit,
    labels = labels_tous_fr
  )
})

output$vif_plot <- renderPlot({
  plot_vif(
    modele_logit,
    labels = labels_tous_fr
  )
})
```

```

    })

    output$tree_graph <- renderPlot({
      tree_graph(
        modele = modele_tree,
        labels = labels_tous_fr
      )
    })

    output$arbre_decision_plot <- renderPlot({
      plot_tree_structure(modele_tree)
    })

    output$rf_graph <- renderPlot({
      rf_graph(
        modele = modele_rf,
        labels = labels_tous_fr
      )
    })

    output$eval_models <- renderPlot({
      evaluer_modeles(
        models = models,
        data_test = data_test,
        variable_cible = "Success",
        positive_class = "Failure"
      )
    })

    output$extreme_num <- renderPlot({
      req(input$vars_extreme_num)
      extreme_plot_num(
        data = data,
        vars = input$vars_extreme_num,
        labels = labels_tous_fr
      )
    })

    output$extreme_quali <- renderPlot({
      req(input$vars_extreme_quali)
      extreme_plot_quali(
        data = data,
        vars = input$vars_extreme_quali,
        labels = labels_tous_fr,
        labels_levels = labels_levels_fr
      )
    })
  }
  shinyApp(ui, server)
}
#-----
# Lancement du dashboard Shiny
#-----
dashboard(
  data = data,
  data_test = data_test,
  data_train = data_train,
  modele_logit = modele_logit,
  modele_tree = modele_tree,
  modele_rf = modele_rf
)

```